The `match_none` query is more useful in scenarios where you want to conditionally exclude all documents, often based on some external logic in your application. For example, if a certain condition in our application is met, we might insert a `match_none` query into the must clause of your `bool` query to ensure that no documents are returned.

Let's say we have an application feature that allows administrators to "lock down" the movie database for maintenance or upgrades. When the database is locked, you want all search queries to return no results rather than throwing erroneous results. In this case, you could insert a `match_none` query based on the *lockdown* condition in your application. The `bool` query written in the snippet below demonstrates such use case:

```
{
  "query": {
    "bool": {
      "must": [{
          "match_none": {}
        }
      ]
    }
  }
}
```

The query gets a `match_none` inserted under specific conditions (database being upgraded) which means, no queries will return results.

In the next section, we learn about the `match` query. Most of the queries we use when working with Elasticsearch are `match` queries in one form or another.

10.5 The match query

The `match` query is the most common and powerful query for multiple use cases. It is a full-text search query returning documents that match the specified criteria. The `match` query can be modified to include many different options.

10.5.1 Format of a match query

Let's first look at the format of the `match` query:

```
GET books/_search
{
  "query": {
    "match": {
      "FIELD": "SEARCH TEXT"
    }
  }
}
```

The query type is match.

The query expects a criterion to be specified as a name-value pair.

The `match` query expects the search criterion to be defined as a field value. The field can be any `text` field in a document whose values are to be matched. The value can be a word or multiple words and can be uppercase, lowercase, or CamelCase.